

LINQ Lab: Movie Manager Application

Overview

In this multi-part lab, students will build a console application called “Movie Manager” that uses LINQ to process a collection of movies. Each part introduces new LINQ concepts and gradually increases the complexity of the application.

Part 1: Basic LINQ Filtering and Sorting

Objective

Familiarize students with LINQ basics, including filtering, sorting, and projecting data.

Tasks

1. Set up a hardcoded list of movies. Each movie should have:
 - **Title** (string)
 - **ReleaseYear** (int)
 - **Genre** (string)
 - **Rating** (double)
2. Filter movies by genre using LINQ `where`.
3. Sort the filtered list by:
 - Release year (descending).
 - Rating (descending).
4. Project the results into a summary of titles and ratings using LINQ `select`.

Example Output

```
Filtered Movies (sorted by year and rating):
```

- ```
1. Mad Max: Fury Road (2015) - Rating: 8.1
2. Die Hard (1988) - Rating: 8.2
```

```
Movie Summary (Title - Rating):
```

- ```
- Mad Max: Fury Road - 8.1
- Die Hard - 8.2
```

Add a screenshot of the console output showing the filtered and sorted movies.

Part 2: Grouping and Aggregation

Objective

Explore advanced LINQ operators like `groupBy` and `aggregate`.

Tasks

1. Group movies by genre using LINQ `groupBy`.
2. Display the count of movies in each genre.
3. Calculate and display:
 - The average rating of all movies.
 - The average rating of movies in each genre.
4. Find the highest-rated movie in each genre.

Example Output

```
Movies grouped by genre:
Action (2 movies)
Comedy (3 movies)
Drama (5 movies)

Average rating of all movies: 7.4
Average rating by genre:
- Action: 8.2
- Comedy: 6.8
- Drama: 7.5

Highest-rated movies by genre:
- Action: Die Hard (8.2)
- Comedy: The Grand Budapest Hotel (8.1)
```

Include a screenshot of the grouped output to help students understand how grouping works.

Part 3: User Input and Dynamic Queries

Objective

Teach students how to build LINQ queries dynamically based on user input.

Tasks

1. Allow users to filter movies by:
 - Release year range.
 - Minimum rating.
2. Let users choose the sort order dynamically (e.g., by year, rating, or title).
3. Use LINQ to build and execute the query based on the input criteria.

Example Output

```
Enter the minimum release year: 2000
Enter the minimum rating: 7.5
Choose sort order (1 - Year, 2 - Rating, 3 - Title): 2

Filtered and Sorted Movies:
1. Inception (2010) - Rating: 8.8
2. The Dark Knight (2008) - Rating: 9.0
```

Add a screenshot of the dynamic query results with user inputs.

—

Part 4: Advanced Projections and Transformation

Objective

Dive into more advanced data manipulation with LINQ.

Tasks

1. Create a new data type `GenreSummary` with:
 - **Genre** (string)
 - **MovieCount** (int)
 - **AverageRating** (double)
 - **TopRatedMovieTitle** (string)
2. Use LINQ to transform the movie list into a collection of `GenreSummary` objects.
3. Display the genre summary.

Example Output

Genre Summary:

- Action: 2 movies, Avg Rating: 8.2, Top Movie: Die Hard
- Comedy: 3 movies, Avg Rating: 6.8, Top Movie: The Grand Budapest Hotel

Include a screenshot of the genre summary output for clarity.

—

Part 5: External Data Sources

Objective

Work with external data sources to make the app more realistic.

Tasks

1. Replace the hardcoded movie list with data loaded from a CSV or JSON file.
2. Use LINQ to filter, sort, and transform the loaded data.
3. Allow users to save filtered results to a new file.

Example Output

```
Loaded 15 movies from movies.json.
```

```
Enter a genre to filter by: Action
```

```
Filtered Movies (saved to filtered_movies.json):
```

1. Mad Max: Fury Road (2015) - Rating: 8.1
2. Die Hard (1988) - Rating: 8.2

Show a screenshot of the console with the file load and save process.

—

Part 6: Asynchronous LINQ

Objective

Introduce the concept of asynchronous programming with LINQ in C#.

Tasks

1. Simulate an asynchronous data fetch operation (e.g., downloading movie data from a REST API).
2. Use `async/await` to process the data with LINQ in the background.
3. Display the results once processing is complete.

Example Output

```
Fetching movie data...

Movies fetched successfully!
Top Movies by Rating:
1. The Dark Knight (2008) - Rating: 9.0
2. Inception (2010) - Rating: 8.8
```

Add a screenshot of the asynchronous data fetch and processing output.

—

Part 7: Unit Testing LINQ Queries

Objective

Introduce the idea of writing testable LINQ queries.

Tasks

1. Refactor the LINQ logic into reusable methods.
2. Write unit tests to verify the behavior of:
 - Filtering by genre.
 - Sorting by year and rating.
 - Grouping by genre.
 - Aggregating average ratings.
3. Use a testing framework like MSTest, xUnit, or NUnit.

Example Unit Test

```
[TestMethod]
public void TestFilterByGenre()
{
    var movies = new List<Movie> { /* Add test data */ };
    var result = MovieProcessor.FilterByGenre(movies, "Action");

    Assert.AreEqual(2, result.Count);
    Assert.IsTrue(result.All(m => m.Genre == "Action"));
}
```

—

Conclusion

This multi-part lab introduces students to LINQ concepts progressively, reinforcing learning at each step. By the end, students will have developed a fully functional and testable movie management app, gaining practical experience with LINQ's power and flexibility.